Caching and Redis

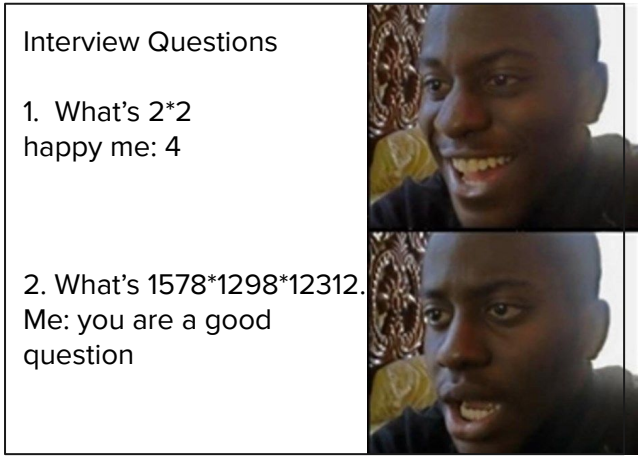
Mohammad Iqbal

Content

- What is Caching?
- Why caching?
- Architecture
- Type of Caching
- In Memory Cache
- Distributed Cache
- Redis
- Redis stand alone
- Redis Sentinel
- Redis Cluster

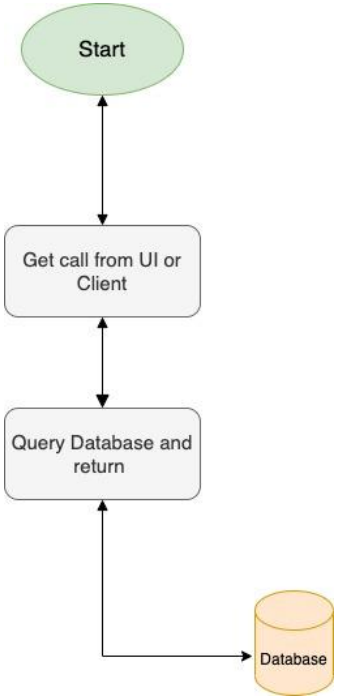
What is Caching?

- A cache is a hardware or software component that stores data so that future requests for that data can be served faster.
- The data will be either a pre-computed data or a result of previous queries.
- The main advantage, and also the goal, of caching is **speeding up** any process API call, page loading etc..
- Increase the overall cost of the application but the trade of between the cost and feature is always beneficial.



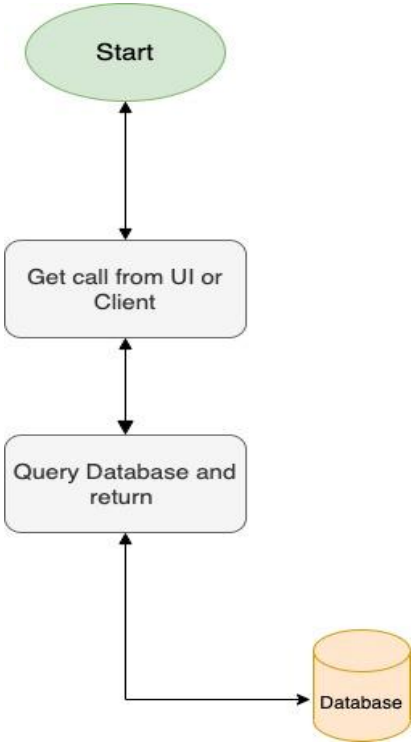
Why Caching?

- Caching data is important because it helps speed up the response time or wait time for user.
- As a large number of queries can be served by cache it reduces overall load on servers which increase efficiency.
- It stores data locally, which means browsers and websites will load faster because access elements such as homepage images have previously been downloaded.

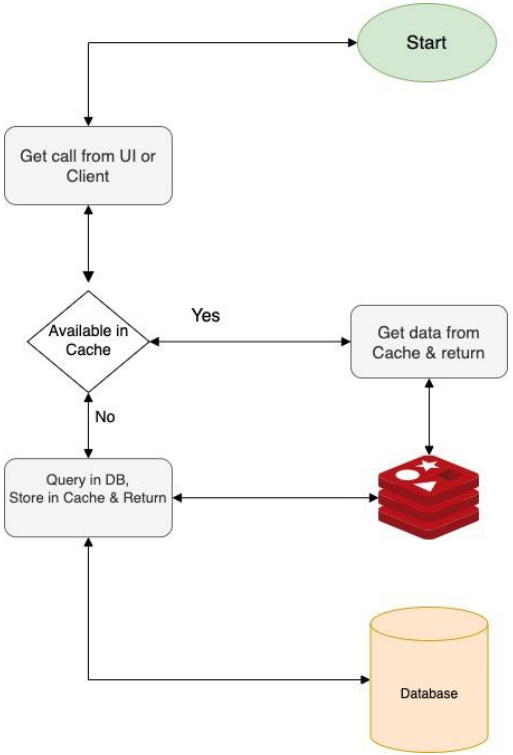


Architecture

Without Cache



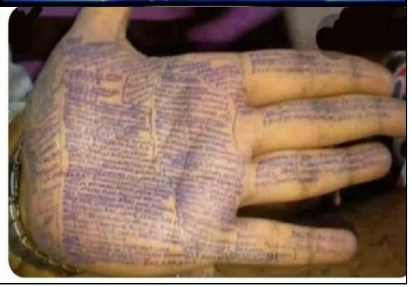
With Cache



Type of Caching

Caching is divided into two part as per the type of storage used.

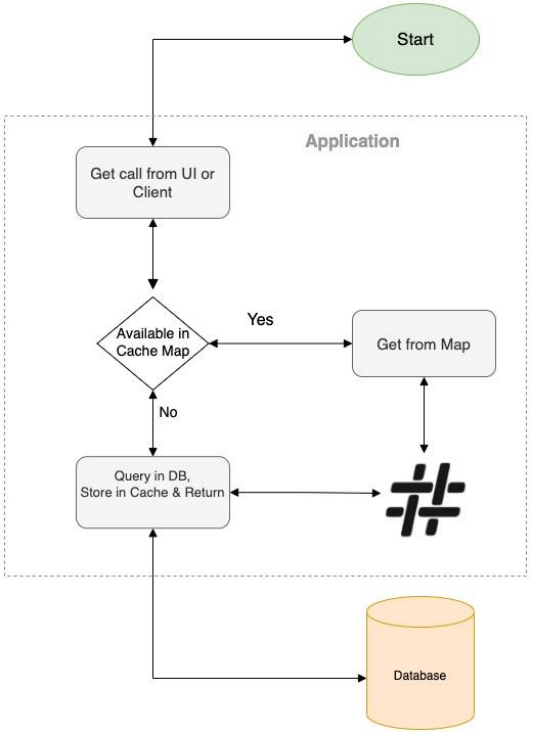
1. In Memory Caching
2. Distributed Caching

In-Memory Cache Question : Name all Indian states 1. Approach one for me: Memorize all the names.	
Distributed Cache 2. Approach : Write down on my hand and read it.	

In Memory Cache

- In memory cache is when we use the application itself as a storage service for cache.

- Usage in memory HashMaps or other data structures to implement
- It is very fast as there are no external network calls.

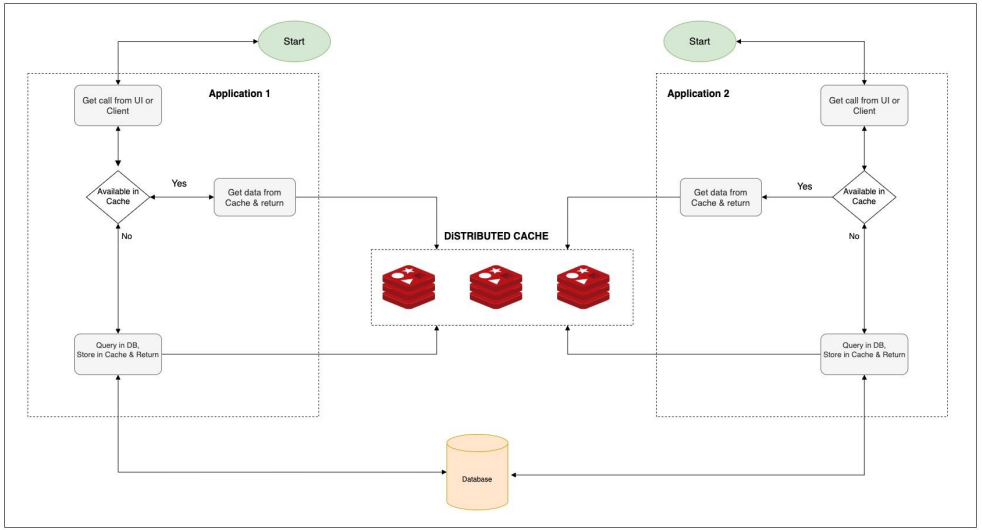


Why not in memory cache?

- Complex to code and maintain
- Application goes down, cache goes down.
- As it usage RAM, the storage is very expensive.
- Requires notifying instances once a change occurs and need to update each instance separately
- Need to load mandatory cache on start of each server
- As each instance has its own cache there might be data inconsistency in the scenario where one instance having different or outdated data than others.

Distributed Cache

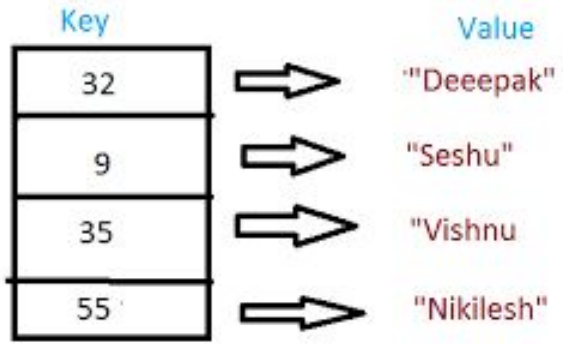
- A cache shared by multiple app servers, maintained as an external service to the app servers that access it.
- Much simpler to maintain
- Even if application goes down cache will be maintained as external data storage is used.
- Very reliable
- Only one place to update
- Assures data consistency
- Examples : **Redis**, ElastiCache, Hazelcast ...



Redis

Redis stands for Remote Dictionary Server

- Redis is a fast, open source, in-memory, key-value data store.
- Redis is written in **C**.
- It supports different kinds of abstract data structures, such as strings, lists, maps, sets, sorted sets,
  HyperLogLogs, bitmaps, streams, and spatial indices.
  Which makes it a better choice than other caching applications.
- It also provide clustering, replication, range queries, **Eviction Policies**.

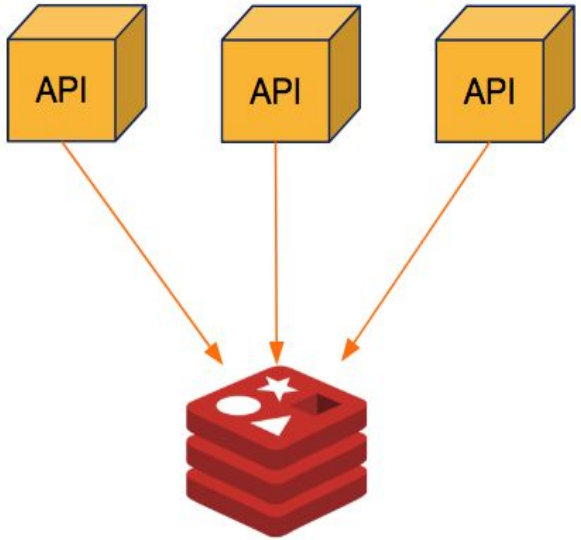


Redis topologies

1. Redis Stand Alone
2. Redis Master-Slave
3. Redis Sentinel
4. Redis Cluster

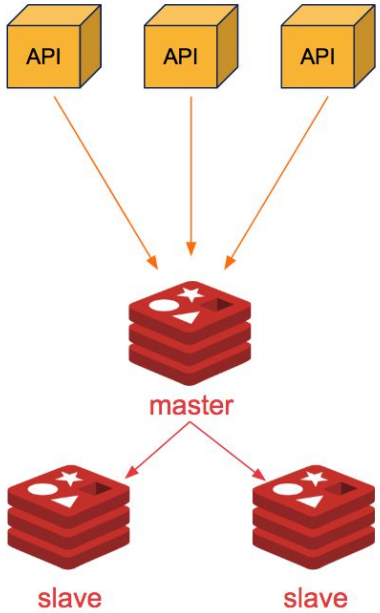
1. Redis stand alone

- A Single Redis node working as the cache.
- Easy to implement and maintain
- Fast, as only one network call, no addition proxy.
- Consistent, as only one node data will always be consistent
- **Single Point of Failure,** node goes down entire cache goes down.
- **No failover**, If node goes down, someone will have to manually restart the redis server.



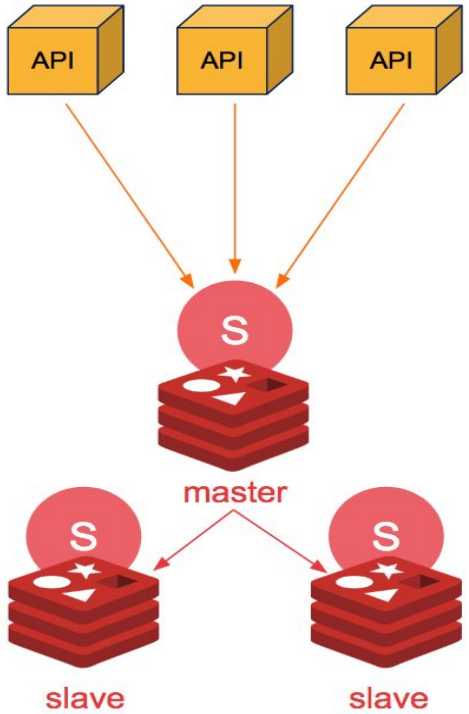
2. Redis Master-Slave

- In this topology multiple slave nodes are attached to one Master slave.
- Data will be updated on this master and then the master will push those changes to its replicas.
- Slaves can talk to the master only, and are only read only nodes.
- In case of node failure:
  - If slave fails : it works as it is. as all calls goes to master only.
  - If master fails : data won't be lost as we have replicas but we will have to **manually reconfigure** and set any of the slave as master
- Client will also have to **update the write node IP** as the master is changes.



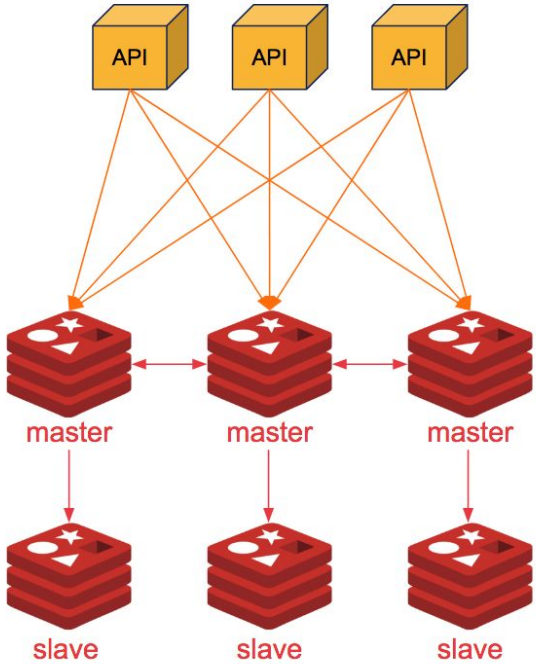
3. Redis sentinel

- Redis Sentinel is a high availability solution provided by Redis, which works on Master-Slave architecture.
- In this topology we have one sentinel node attached to each redis node. The Sentinel decide the master node or we can specify as well in first iteration.
- We can additionally specify to redirect all reads to slaves which decrease load on Master and increase efficiency.
- **Availability** is extremely high.
- **Automatic failover.** If a master node fails, Sentinel starts the failover process where a replica is promoted to master, the other additional replicas are reconfigured to use the new master.
- **Monitoring**. Sentinel constantly checks if your master and replica instances are working as expected.
- **Notification**. Sentinel can notify the system administrator, or other computer programs, via an API, that something is wrong with one of the monitored Redis instances.
- If we have a large number of keys, it doesn't solve **horizontal scaling**.



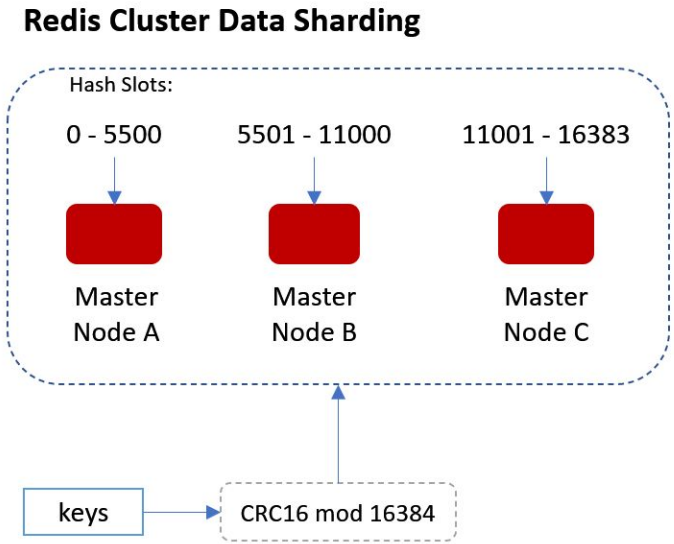
4. Redis cluster

- Redis cluster solves the horizontal scaling problem, where data is **sharded** among multiple master Nodes.
- **Replication** : Each master can have as many slaves as required upto 1000. Min 1 slave is required.
- Must have at least 6 Redis nodes – 3 for masters and three for slaves.
- There are two major benefits.
    - If master fails, 66% of application will face 0 down time.
    - After the Failover is completed the slave will become master and it will start working as it was.



4. Redis cluster

- Main purpose of the Redis cluster is to equally/uniformly distribute your data load by sharding
- Redis Cluster does not use consistent hashing, but a different form of sharding where every key is conceptually part of what is called as hash slot
- There are 16384 hash slots in Redis Cluster, Every node in a Redis Cluster is responsible for a subset of the hash slots, so, for example, you may have a cluster with 3 nodes, where:
  Node A contains hash slots from 0 to 5500, Node B contains hash slots from 5501 to 11000, Node C contains hash slots from 11001 to 16383
- This allows us to add and remove nodes in the cluster easily. For example, if we want to add a new node D, we need to move some hash slot from nodes A, B, C to D
- Redis cluster supports the master-slave structure, you can create slaves A1,B1, C2 along with master A, B, C when creating a cluster, so when master B goes down slave B1 gets promoted as master



Redis Sentinel vs Cluster

Redis Sentinel is a good option for a smaller implementation with high availability concerns. On the other hand, Redis Cluster is a clustering solution that handles the scanning for larger implementations.

Metric	Redis Sentinel	Redis Cluster
Architecture	Better visibility to the users.	Data sharding, maintains happens internally
Availability	Availability is high here as we the system can stand totalNumberOfNodes-1 failures. Additionally as we have Sentinel nodes to monitor the nodes and decision making, makes it more available.	Even if we have 6 nodes in our system, if one master slave pair goes down we will lose a lot of data. To Erase it we will require more number of slave nodes per master. Increase cost.
Cost	Cheaper as you can start with only 2 nodes.	Expensive as it requires min 6 machines.
Scalability	We can vertically scale by increasing the size of master node but we have a certain limitation to that as well.	Scalability is great here as we can add as many master as we want.
Data Sharding	No data sharding, as each node (master or slave) have the complete data copy.	Data is sharded among number of partitions created or number of master slave groups.


Thank you!